

Example: Monitors

declare a shared object.

A monitor is a shared object.

```
abstract class Monitor {  
private:  
    mutex m;  
    condition_variable cv;  
public:  
    void Lock()          { m.lock(); }  
    void Unlock()        { m.unlock(); }  
    void Wait()          { cv.wait(m); }  
    void SignalOne()      { cv.signal(); }  
    void SignalAll()      { cv.broadcast(); }  
}
```

Example: Monitors

不能创建实例。

abstract class,

- A monitor class may derive from *Monitor* and use the *Monitor* operations.
- Alternatively, a monitor class could simply have a *Monitor* instance in the class and use the *Monitor* operations through this instance. BUT...
 - Having more than one *Monitor* instance may lead to incorrect usage, so derivation is a better approach than using an instance.

Parallel Programming in C#

- We will look closely at the parallel programming features of C#
- C# has a reasonably long history, and these features grew and evolved over the time.

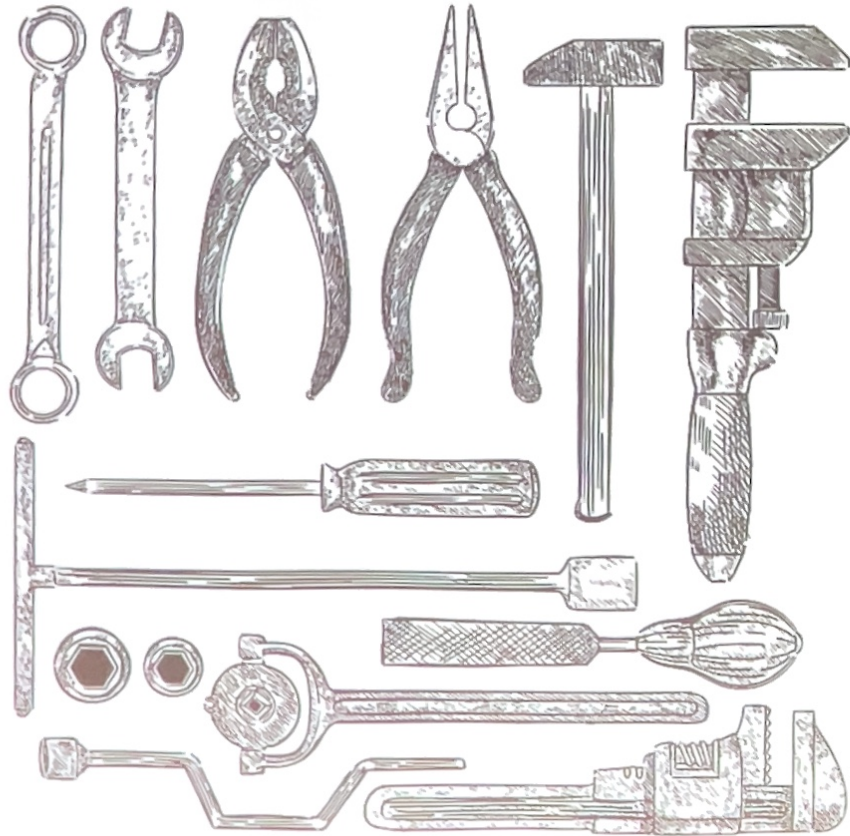
Parallel Programming in C#

- We will look closely at the parallel programming features of C#
- C# has a reasonably long history, and these features grew and evolved over the time.

Parallel Programming in C#

- A lot of the API is bundled up in the so-called Task Parallel Library (TPL) that is part of the .NET framework. The goal of TPL is to simplify writing parallel programs.
- While they are a higher level mechanisms to simplify writing parallel programs, a good understanding of concepts underlying multi-threaded programming (e.g., mutexes, condition variables, race conditions, deadlocks, etc.) is necessary.

Parallel Programming in C# – A Big Toolset



Library: *BackgroundWorker, Barrier, Interlocked, Monitor, Mutex, ReaderWriterLock, Semaphore, SemaphoreSlim, SpinLock, SpinWait, Thread, ThreadPool, ThreadStart, Parallel, Parallel.For, Parallel.Invoke, Task, Task.Wait, TaskFactory, ValueTask, PLINQ, etc.*

C# Keywords: *async, await, lock, volatile*

Why so many?

- Some of the primitives are named so that they can be used inter-process. Others are unnamed and therefore they can only be used intra-process (i.e., between threads of the same process).
- Some primitives use spin waits (e.g., *SemaphoreSlim*) so they are optimized for short wait times.
- Some are very specific (e.g., *BackgroundWorker* used in UI, *PLINQ* used with queries).
- You need to know your toolset for the best use of the tools.

The volatile keyword

- The volatile keyword indicates that a field might be modified by multiple threads that are executing at the same time.
- The compiler, the runtime system, and even hardware may rearrange reads and writes to memory locations for performance reasons.
- Fields that are declared volatile are excluded from certain kinds of optimizations.
- There is no guarantee of a single total ordering of volatile writes as seen from all threads of execution.

The volatile keyword

- Volatile memory operations are for special cases of synchronization, where normal locking is not an acceptable alternative.
- Under normal circumstances, the C# lock statement and the Monitor class provide the easiest and least error-prone way of synchronizing access to data.

The lock keyword

- The lock statement acquires the mutual-exclusion lock for a given object, executes a statement block, and then releases the lock.
- Any other thread is blocked from acquiring the lock and waits until the lock is released.

The lock keyword

```
lock (x)
{
    // Your code...
}
```

Note: x above must be a reference type; typically we use an **object**. This object might be this.



```
object __lockObj = x;
bool __lockWasTaken = false;
try
{
    Monitor.Enter(__lockObj, ref __lockWasTaken);
    // Your code...
}
finally
{
    if (__lockWasTaken)
        Monitor.Exit(__lockObj);
}
```

Task

- Task represents an asynchronous operation. It comes in two flavours: Task that does not return a value, and Task<TResult> which returns a value (of type TResult).
- The most important (static) method of the Task is Run which points to a function that specifies the work to run on the ThreadPool and returns a Task or Task<TResult> handle for that work.
- The return handle allows us to wait for the task's completion and to consume results.

```
var t = Task.Run(DoSomeLongJob);  
var t2 = Task.Run(() => DoSomeOtherLongJob("Scene36", 42));
```

The async keyword

- Use the async modifier to specify that a method, lambda expression, or anonymous method is asynchronous.
- If you use this modifier on a method or expression, it's referred to as an async method.
- An async method runs synchronously until it reaches its first await expression, at which point the method is suspended until the awaited task is complete.
- In the meantime, control returns to the caller of the method.

Async/Await

配合 task< 使用

task1 $\longleftrightarrow$ task2

等待时挂起, 执行

其它程序, 完成后返回

await

```
public static async Task Main()
{
    Task<int> downloading = DownloadDocsMainPageAsync();
    Console.WriteLine($"Launched downloading.");

    int bytesLoaded = await downloading;
    Console.WriteLine($"Downloaded {bytesLoaded} bytes.");
}

private static async Task<int> DownloadDocsMainPageAsync()
{
    Console.WriteLine($"About to start downloading.");

    var client = new HttpClient();
    byte[] content = await client.GetByteArrayAsync("https://docs.microsoft.com/en-us/");

    Console.WriteLine($"Finished downloading.");
    return content.Length;
}
```

```
About to start downloading.
Launched downloading.
Finished downloading.
Downloaded XXXXX bytes.
```

Further Work

- Go through the parallel programming documentation at Microsoft online.
- Compile and run the async/await example. Make sure you understand the logic and control flow.